FerixCommerce — Full Project Code Review

Review date: 2026-10-24
Repository: `Asaphis/FerixCommerce`
Revision reviewed: `58ad0ab9f10db4641d95499c8ca0bfbfd8911367`
Review type: Read-only static code and architecture review
Source changes made: None

1. Executive conclusion

FerixCommerce is a substantial commerce platform implementation, not an empty prototype. It contains a FastAPI backend, PostgreSQL-oriented data layer, three production-shaped Next.js applications, a unified UI preview, CMS tooling, product placement records, seller workflows, admin review workflows, tests, and CI configuration.

However, it is **not ready to be considered reliable or production-safe**. The main problem is not that the project uses a document for CMS pages. The main problem is that the system has several competing sources of truth and several write paths with different rules:

- Some CMS edits update one section and create a draft.
- Other CMS operations read and rewrite the whole page.
- Information-page content can be written directly into live data.
- Page titles can become public before the rest of a draft is published.
- Product assignments are relational, but bulk replacement can delete assignments that the form cannot display.
- Admin preview and storefront rendering do not use the same normalized section model.
- Seller, admin and marketplace apps duplicate API/session logic.
- Backend authorization and validation are incomplete on multiple read and write routes.

These inconsistencies explain the user-visible symptoms: changes appearing to revert, preview showing a placeholder instead of the real hero/slider, layouts saving without changing the storefront, sections disappearing silently, and product or placement settings becoming difficult to clear.

The versioned document approach can remain. It needs one canonical command layer, typed validation, optimistic concurrency, one draft/publish boundary, and shared rendering contracts.

2. Repository map

```
FerixCommerce/
├── Backend/                    FastAPI service and database logic
│   ├── app.py                  Shopper API, auth, cart, checkout, public
│   ├── core.py                 Models, migrations, persistence, shared ru
│   ├── routers/admin.py        Admin catalog, CMS, taxonomy, media, opera
│   ├── routers/merchant.py     Seller products, orders, inventory, media
│   ├── cms_sections.py         Section-family and scheduling rules
│   ├── notifications.py        Email templates and notifications
│   ├── ratelimit.py            Request throttling
│   └── tests/                  Backend contracts and regression tests
├── WebPhase/
│   ├── marketplace/            Shopper marketplace and public storefront
│   ├── merchant/               Seller workspace
│   ├── admin/                  Platform administration and CMS
│   ├── shared/                 Shared source for the three apps
│   └── sync-shared.mjs         Copies/checks shared UI files
├── UiUxPreview/                Unified in-memory UI/UX preview
├── Prototype/                  Earlier static HTML prototype
├── BackendMock/                Mock backend/demo implementation
├── tools/                      Maintenance scripts
└── *.md                        Product specifications, audits and reports
```

The repository contains approximately 383 TSX files, 48 TypeScript files and 27 Python files in the main application areas, plus separate copies of UI primitives and API/session utilities across the three applications.

3. Current technology stack

Backend

- Python 3.11-compatible code
- FastAPI 0.141.1
- SQLAlchemy 2.0.44
- PostgreSQL via `psycopg`
- Pydantic 2.13.5

- Uvicorn
- Optional Cloudinary and Resend integrations

Frontend

- Next.js 16.4.0 in Marketplace, Merchant and Admin
- React 19.2.3
- TypeScript
- Tailwind CSS 4
- Radix UI
- React Hook Form
- Zod
- Recharts
- Lucide React

Build and CI

- Independent `npm ci`, typecheck and build per application
- GitHub Actions matrix for all three WebPhase apps
- PostgreSQL CI service
- Backend unit/contract tests and migration smoke tests

The technology choices are appropriate for the product. Replacing the whole stack is not the priority. Correctness, shared contracts, authorization and workflow consistency are the priority.

4. What is already sound

The following architectural decisions are reasonable and should be preserved or strengthened:

1. A versioned page document is suitable for storing an ordered, flexible page layout.
2. `SectionPlacement` records are the correct direction for product membership, placement-specific price, quantity, labels and scheduling.
3. Public product serialization is allowlisted rather than exposing every internal catalog field.
4. Seller product approval is conceptually separated from public marketplace visibility.

5. The backend has explicit admin and merchant routers rather than putting all logic in frontend code.
6. The repository has meaningful backend tests covering several CMS, product lifecycle, seller ownership and public-filtering cases.
7. The production merchant UI has thoughtful responsive table/card behavior, even though it still needs browser-level verification.
8. The CI workflow installs locked dependencies and attempts typecheck/build gates for all three applications.

These strengths are not enough to certify the project. They provide a good base for repair.

5. Critical findings and required corrections

P0-A — Authorization is incomplete on admin and merchant read routes

Evidence: `Backend/routers/admin.py` routes such as `/admin/users`, `/admin/orders`, `/admin/analytics`, `/admin/payments` and audit reads authenticate staff but do not consistently enforce the corresponding permissions. Merchant read routes such as payouts, analytics, customers, orders and media similarly authenticate a merchant staff session but omit the declared read permissions.

Impact: A staff account with a restricted permission list may still read customer PII, orders, payment projections, analytics or merchant data outside its intended scope. This is a least-privilege failure.

Correction: Create a route authorization matrix. Every read and write route must declare and enforce a permission such as `cms.manage`, `orders.view`, `payments.view`, `merchant.orders.view` or `merchant.analytics.view`. Add tests for every role and custom permission combination, not only the owner fixture.

P0-B — Session tokens are accepted and sent in URLs

Evidence: `Backend/app.py:266-297` copies `session` from query/body into a request header. `WebPhase/merchant/lib/api.ts:388-393` appends session tokens to GET URLs, and upload URLs also use query sessions.

Impact: Tokens can enter proxy logs, browser history, tracing systems, cache keys and referrer headers. Conflicting header/query/body credentials can also create ambiguous behavior.

Correction: Move authenticated calls behind a server-side BFF or HTTP-only session exchange. Require an Authorization/header transport. If a temporary upload compatibility path

remains, use short-lived scoped tokens, redact query parameters, reject conflicts and set a removal deadline.

P0-C — CMS draft/publish semantics are inconsistent

Evidence: Section-field editing uses a draft path, but information-page content writes directly to `ContentDocument.data['content']`. Page title changes can be written before the draft version is published. Legacy section add/remove paths mutate page data directly. Public reads therefore do not have one consistent publication boundary.

Impact: The user cannot predict whether Save means “private draft” or “live now.” Content may appear partially published.

Correction: Put title, page metadata, section order, visibility, section fields and information-page content into one versioned draft snapshot. Only the publish transaction may update the public snapshot. Remove or harden alternate direct-mutation endpoints.

P0-D — CMS writes have no reliable optimistic concurrency

Evidence: Whole-page add/remove/reorder/visibility operations fetch a page, modify an array and write it back. Draft/publish version calculation is based on existing length and has no expected-version or ETag precondition.

Impact: Two tabs or operators can silently overwrite each other. A late request can make a change appear to revert.

Correction: Add a monotonically increasing revision and require `expectedRevision` or `If-Match` on every CMS command. Return `409 Conflict` for stale writes. Use section-scoped commands for add, patch, move, visibility and remove. Lock/atomically update the document and create a unique version record.

P0-E — Seller submission can write under the wrong catalog key

Evidence: `Backend/routers/merchant.py:1753-1771` writes a submission with the product ID, while canonical catalog rows are keyed by slug through `put_row`. Other product paths use the slug.

Impact: Submission may create a second catalog row instead of updating the canonical product. Review status and public filtering can then operate on different records.

Correction: Resolve the canonical row and always write using the canonical key. Add a migration/cleanup for duplicate ID-keyed product rows and a regression test for submission, review and public invisibility.

P0-F — Checkout/inventory architecture is not safe to enable yet

Evidence: Cart checks stock but does not reserve it. Checkout placement currently returns unavailable. Merchant stock adjustments read and write JSON without row locking or compare-and-swap.

Impact: Enabling checkout without a transaction boundary would allow overselling and lost inventory updates.

Correction: Implement checkout as an idempotent transaction: validate every line, lock/version inventory, reserve/decrement stock, create an order, create a provider payment intent and store provider IDs. Release reservations on expiry/cancel/refund. Add concurrency and retry tests before enabling real checkout.

6. CMS, sections and rendering findings

High — Admin preview does not use the same hero/slide model as storefront rendering

The storefront can render saved hero slides as a carousel, while Admin preview may show a placeholder or a single selected image. Direct-media heroes also synthesize `showButtons: true`, and `MediaHero` does not receive the saved `showButtons` setting.

Correction: Create a shared section normalization package. Both Admin preview and Marketplace render the same normalized `HeroSection`, `Slide`, CTA and media contract. Preview must use the same component or a shared pure renderer, not a separate approximation.

High — Flash-sale controls do not map to output

Admin values such as carousel/grid/rows/across are not consistently interpreted by `flash-sale.tsx`; the storefront mainly switches behavior for a literal `horizontal` value and fixes responsive columns by breakpoint. A setting can therefore save successfully without changing the output.

Correction: Define an enum once, validate it on the server and map every enum to observable behavior. Add contract tests for every section family and setting.

High — Product section layout and hero text-width controls can be save-but-no-effect

The product section display layout is saved, but the public renderer may use section type instead of the saved layout. Hero text width is not consistently passed to direct-media or carousel rendering.

Correction: Remove controls that are not implemented or wire them through a shared normalized configuration adapter. Every editable setting must have a test proving its effect.

High — Configured sections can silently disappear

`render-sections.tsx` returns `null` when product placements or advert feeds are empty. Home fallback logic checks whether section records exist rather than whether any section actually rendered.

Correction: Distinguish configured, eligible, empty and failed states. Admin should show the same reason the live renderer would omit a section. Public pages should either show a deliberate empty state or omit the section by an explicit contract, never accidentally.

High — Product placement bulk replacement is weaker than single placement save

The bulk route deletes existing placement rows and rebuilds them without all the numeric/date validation used by the single-placement route.

Correction: Use one placement service and validator for both paths. Validate every row before deleting anything. Commit atomically or reject the complete request with field-level errors.

High — Product edit can delete placements the form cannot display

The form shows only currently assignable sections. Bulk replacement deletes every existing row and rebuilds only submitted checkboxes. A hidden, unavailable or draft-removed section cannot be preserved.

Correction: Use patch semantics, or include unavailable existing placements as retained records with an explicit remove action. Show a destructive-removal confirmation.

High — Product and placement saves are separate commits

Admin product details save first and placement synchronization follows in another request.

Correction: Add a transactional `saveProductWithPlacements` command. If separate APIs are retained, use an idempotency key, explicit partial-success state and retry only for the failed operation.

Medium — Empty/expired window behavior is unclear

Section and placement end checks use `moment > ends`, keeping an item live at the exact end instant. Page-level status does not explain that a section is scheduled or expired.

Correction: Use a consistent half-open interval, normally `startsAt ⩽ now < endsAt`, normalize time zones and show scheduled/expired diagnostics in Admin and preview.

Medium — Ordering has two authorities

Some paths reorder array entries without renumbering `section.position`, while public non-homepage rendering sorts by position.

Correction: Make the backend reorder command atomically renumber positions and return the persisted order. All readers must use that one authority.

Medium — Banner updates can reactivate campaigns or retain stale media modality

An omitted `active` value may default to true during an update. Replacing a video with an image can leave the old `videoUrl` populated.

Correction: Preserve existing active state when omitted. Use explicit activate/deactivate commands. Store one canonical media object or clear the opposite modality field when kind changes.

Medium — Information-page saves accept arbitrary content

The information-page endpoint bypasses versioning and accepts arbitrary content without a clear schema, size limit or URL policy.

Correction: Validate by page type, cap content size, sanitize/allowlist links and route through the normal draft/publish pipeline.

7. Catalog, taxonomy and product workflow findings

High — Seller price/stock validation is incomplete at the backend boundary

Seller create/update paths convert values but do not consistently enforce nonnegative finite price, compare-at and stock rules. Browser checks are not a security boundary.

Correction: Add Pydantic request models with finite/range validation, cross-field rules and unknown-field rejection. Apply the same schemas to Admin placement terms.

High — Seller compare-at price cannot reliably be cleared

The form sends `null`, but backend update logic treats `None` as “do not update.” Similar omission semantics make blank placement dates, labels and colors stick.

Correction: Distinguish omitted from explicit null/empty using typed patch models and `model_fields_set`. Define clear behavior for every optional field.

High — Product edit can change category accidentally

Admin category options are derived from categories found on products instead of the managed taxonomy registry. If the current category is absent, a browser select can choose the first available option when an unrelated field is saved.

Correction: Load categories from the taxonomy manager. Preserve the current orphaned value as a retained/disabled option and require explicit reassignment.

Medium — Taxonomy references are not consistently validated

Admin product category/collection fields and seller collections can be accepted without confirming managed records. Collection deletion does not consistently guard in-use references.

Correction: Validate category, brand and collection references against active taxonomy on every write. Prevent deletion of in-use records or explicitly migrate/repair references. Use soft deletion for taxonomy.

Medium — Public facets can expose hidden taxonomy/store records

Public product rows may be filtered while facet arrays enumerate hidden categories or stores.

Correction: Use one `public_taxonomy` and `is_public_store` predicate for every public facet and add response-shape tests.

Medium — Product image retention/removal is ambiguous

Admin product editing mixes existing image values, new uploads and removal fields without an explicit retained-image list.

Correction: Submit `retainImage[]` separately from new uploads and compute the final image set server-side. Preview the resulting set before commit.

Medium — Product identity is inconsistent in merchant routes

Merchant list links use product IDs while the dynamic page and revalidation use a slug-shaped parameter.

Correction: Choose one stable internal identifier and one public route identifier. Keep API lookup, links and cache invalidation consistent.

Medium — Upload-then-create is not transactional

Seller media uploads occur before product creation. A later failure leaves orphan assets, and retries can duplicate uploads.

Correction: Use staged uploads tied to an upload session/product draft, commit/rollback,

content hashes and idempotency keys.

8. Marketplace and public rendering findings

High — Storefront and Admin preview are separate implementations

Store pages render static catalog/about/recommendation areas before CMS sections, while Admin treats the CMS document as the page surface. Reordering or disabling CMS sections therefore cannot fully control store composition.

Correction: Define whether CMS is additive or authoritative. Then use the same ordered page model and renderer in both preview and live storefront.

High — Brand not-found pages can become false 200 pages

`brands/[slug]` treats API failure or unknown brand as a valid page and falls back to generic explore content.

Correction: Distinguish outage from not-found. Call `notFound()` for absent brands when catalogue data is available and provide canonical metadata.

Medium — CMS visibility values are not normalized consistently

Some code understands string booleans while other renderers use strict boolean checks. A serialized `"false"` can render as visible.

Correction: Normalize all booleans, enums, dates and numeric settings once at the API boundary. Renderers should receive typed values only.

Medium — Public data fetching is serial and duplicated

Explore/listing flows fetch products, CMS, adverts and home data sequentially; layouts fetch categories separately.

Correction: Parallelize independent requests, memoize public requests and use revalidation/cache tags. Fetch only data required by a section.

Medium — Shipping and returns copy is hard-coded

Listing/policy pages contain fixed values such as free delivery over `$120` and 30-day returns while other areas read backend settings.

Correction: Expose effective policy settings from one API and render them everywhere. Avoid hard-coded currency/region assumptions.

Low/Medium — Images and carousels need accessibility/performance work

Raw `<img>` usage lacks consistent dimensions and responsive optimization. Hero carousel controls are not keyboard-complete, inactive slide focus management is incomplete, and progress is not announced.

Correction: Use `next/image` with explicit dimensions and remote allowlists, prioritize the lead hero, add labelled previous/next/pause controls, indicators, `aria-current`, and inert/focus-safe inactive slides.

Low — SEO metadata is incomplete

Product/category/collection/brand/search routes lack consistent canonical, Open Graph and indexing rules. Sitemap coverage is limited and omits several entity families.

Correction: Add route-specific metadata, canonical URLs, OG images, noindex rules for search/filter variants and paginated sitemap retrieval for published products, collections and brands.

9. Merchant panel and shared architecture findings

High — Merchant product writes are over-postable and under-validated

Server actions accept many unchecked values including arrays, URLs, stock, shipping, channels and image lists. Product actions repeat policy rather than using one schema.

Correction: Use shared Zod/Pydantic contracts, reject unknown fields and make backend ownership/status/channel rules authoritative.

High — Merchant edits forcibly alter marketplace state

`saveProductAction` sets pending review and marketplace enabled on every edit, even when the seller is editing a store-only or already-approved product.

Correction: Separate content edits from channel submission. Preserve channels unless explicitly changed. Use a server-side product state machine.

High — Order transitions are stringly typed

The UI warns that cancellation is not a refund, but action payloads accept arbitrary fulfillment strings and do not expose a complete server-side transition state machine.

Correction: Define typed transitions: paid → processing → shipped → delivered; cancellation/refund as separate authorized commands; tracking required for shipment; all

changes audited.

Medium — Session refresh is not persisted centrally

Merchant API refreshes an expired token but does not consistently write the replacement into the cookie, causing repeated refresh and races.

Correction: Centralize session refresh in a BFF/server action and rotate the cookie atomically.

Medium — API/session/env logic is duplicated across all apps

Marketplace, Merchant and Admin each contain local API, action, session and environment patterns.

Correction: Create a private workspace package containing API client, error model, auth/session handling, normalized public contracts and shared domain types. Keep role-specific commands in each app.

Medium — Shared UI is copied rather than packaged

`sync-shared.mjs` copies UI files and only checks drift when explicitly run. Production and preview also have separate button, pill, table and status systems.

Correction: Create a small tokenized design-system package, or make shared drift checking a required CI gate. Avoid three physical copies of security-sensitive or accessibility-sensitive UI primitives.

Medium — Merchant UX still needs workflow cleanup

- Product list has no visible cursor pagination.
- Product image picker replaces the selected file list when used again.
- Uploads lack progress, MIME/dimension checks, reorder controls and robust retry behavior.
- Delete does not visibly require confirmation or prefer archive semantics.
- Inventory adjustment needs a ledger and conflict/version response.
- Orders and inventory tables have large minimum widths that need tablet testing.
- Navigation has overlapping customer/audience concepts and too many destinations in a More drawer.
- Broad global CSS selectors and `!important` make component styling fragile.

UiUxPreview boundary

`UiUxPreview` is explicitly in-memory and non-persistent. It should not be mistaken for a production implementation. Some simulated actions imply effects that do not match the real backend, including refund behavior and direct inventory mutation.

Correction: Label every simulated surface clearly, use a mock adapter with production-shaped commands, and mirror real semantics even in a prototype.

10. Recommended target architecture

A. Canonical CMS command API

Create a single service with commands like:

```
createPage
savePageDraft
patchSection
addSection
moveSection
setSectionVisibility
removeSection
savePageMetadata
saveInformationPageContent
saveProductPlacements
publishPage
```

Every command should:

1. Authenticate the actor.
2. Enforce the required permission.
3. Load the current document revision.
4. Validate the page family and section schema.
5. Validate related taxonomy, media and placement references.
6. Require `expectedRevision`.
7. Write a new draft snapshot transactionally.
8. Write an audit event.

9. Return the new revision and a normalized document.

B. One source of truth per concern

- Page layout/order/visibility/settings: versioned page draft document.
- Product identity/pricing/inventory/ownership: catalog/product records.
- Product-to-section membership and placement terms: validated `SectionPlacement` records.
- Public presentation: published page snapshot plus currently eligible placement rows.
- Media: media asset records with validated canonical URLs and ownership.

Do not keep multiple mutable representations of the same setting unless one is explicitly materialized and rebuilt transactionally.

C. Shared normalized section contract

Define typed schemas for all section families:

```
HeroSection
PromoBannerSection
FlashSaleSection
ProductGridSection
ProductCarouselSection
CategorySection
BrandSection
CollectionSection
AdvertSection
InfoSection
```

The backend returns normalized values: booleans are booleans, dates are normalized, enums are canonical, URLs are validated, and absent/null semantics are defined. Admin preview and Marketplace consume the same contract and shared pure renderer.

D. Explicit state machines

Use explicit state transitions for:

- Product lifecycle: draft → pending_review → approved/rejected → archived.
- Marketplace channel lifecycle: disabled → requested → enabled/disabled.

- Order lifecycle: pending_payment → paid → processing → shipped → delivered; cancellation/refund separate.
- Media lifecycle: staged → attached → published → orphaned/removed.
- CMS lifecycle: draft → previewable → published → superseded.

Do not allow arbitrary strings from forms to mutate these states.

E. BFF/session model

Each Next.js app should use a server-side API client or shared BFF. The browser should not carry long-lived backend session tokens in URLs. The BFF should normalize errors, refresh sessions once, set secure HTTP-only cookies and prevent cache leakage.

F. Observability and user feedback

Every mutation response should include:

```
success
entityId
revision
changedFields
warnings
fieldErrors
nextAction
```

Admin should show whether a change was saved as draft, published, scheduled, rejected or partially failed. Preview should show section eligibility diagnostics in non-public mode.

11. Prioritized implementation plan

Phase 0 — Protect data and permissions

1. Remove session tokens from URLs and establish a secure BFF/session transport.
2. Enforce permissions on every admin and merchant read/write route.
3. Add backend range/schema validation for product and placement writes.
4. Fix canonical seller submission key.
5. Add strict media/banner/brand URL validation.

6. Prevent unknown taxonomy references and protect in-use taxonomy deletion.
7. Escape notification HTML.

Phase 1 — Make CMS reliable

1. Unify all CMS changes under draft-only commands.
2. Add revision/ETag checks and `409` conflicts.
3. Make title and information-page content part of the draft.
4. Replace whole-page reorder/visibility/add/remove writes with section-scoped commands.
5. Make placement saves transactional and patch-safe.
6. Define explicit null/clear semantics.
7. Normalize section positions and scheduled windows.

Phase 2 — Make preview equal storefront

1. Create the shared normalized section contract.
2. Reuse section-family adapters/components in Admin preview and Marketplace.
3. Fix hero slides, CTA visibility, media kind and text-width propagation.
4. Make flash-sale layout values map directly to renderer behavior.
5. Add eligibility diagnostics for empty, hidden, pending, expired and unassigned sections.
6. Define whether store CMS is additive or authoritative and implement one rule.

Phase 3 — Harden catalog and seller operations

1. Build product create/update/submission around typed commands.
2. Separate product content updates from marketplace channel submission.
3. Add staged/idempotent media upload.
4. Add archive/confirmation semantics instead of unsafe hard deletion.
5. Implement inventory ledger/versioning and order transition validation.
6. Use one taxonomy manager source for categories, brands and collections.
7. Add product/placement transaction and retry behavior.

Phase 4 — Consolidate frontend architecture

1. Extract shared API/session/error/domain packages.

2. Replace copied UI primitives with a tokenized design system.
3. Fix responsive tables/navigation and accessibility.
4. Add `next/image`, image dimensions, lazy/priority policies and media allowlists.
5. Complete canonical metadata, sitemap and not-found behavior.
6. Keep `UiUxPreview` explicitly mock-only and align its commands with production semantics.

Phase 5 — Verification required before release

Add automated coverage for:

- Permission matrix across all admin/merchant reads and writes.
- CMS draft/publish isolation for every page family.
- Concurrent edits and stale revision rejection.
- Preview/storefront parity for every section family and editable setting.
- Product placement preservation and explicit removal.
- Seller price/stock/compare-at validation and clearing.
- Taxonomy references and deletion-in-use.
- Media URL scheme/host validation.
- Session leakage and conflicting credential transports.
- Inventory concurrency, checkout idempotency and payment webhook linkage.
- 320px, 375px, 768px and 1280px browser snapshots.
- Keyboard and screen-reader behavior for navigation, forms and carousels.

12. Verification notes

The review itself was static and read-only. No production service or production database was accessed.

- `python3 -m compileall -q Backend` passed.
- A direct test invocation from the repository root was not a valid full test run: the command lacked the Backend working directory and the sandbox did not have all Python dependencies installed. The repository CI workflow installs dependencies and runs tests from `Backend`.

- No frontend dependency installation or browser visual verification was used as evidence for the findings.
- The Git worktree remained clean after the review.

Passing a build or existing test suite would not remove the code-level findings above. The priority is to repair the contracts and workflow boundaries, then add tests that prove the user-visible behavior.

13. Final answer to the main design question

Should FerixCommerce stop using a document for CMS pages?

No. A versioned document is a reasonable representation for an ordered page layout. The project should keep it for page structure, but stop treating it as an uncontrolled mutable JSON blob.

The reliable design is:

- typed section schemas;
- section-scoped commands;
- relational placement records for products;
- one draft/publish boundary;
- revision checks;
- one normalized contract for Admin preview and storefront;
- strict backend validation and permissions;
- explicit state machines for products, channels, orders and media.

That approach directly addresses the reported “it saved then went back,” “preview does not look like the live page,” “the setting saves but has no effect,” and “product assignment disappeared” problems without requiring a complete rewrite of the stack.